SC2001 Lab 2
Dijkstra's Algorithm

- Wang Yongqing
- Johannes Yap
- Ong Jee Shen
- Balodi Shalok

SDDB Group 6

Dijkstra A

```cpp
// part a
// stored: adjacency matrix, pq: array
vector<int> dijkstra_a(vector<vector<int>> &graph, int src)
{
    int V = graph.size();
    vector<int> dist(V, INT_MAX);
    vector<bool> visited(V, false);

    // Distance from source to itself is 0
    dist[src] = 0;

    // Repeat V times
    for (int count = 0; count < V; count++)
    {
        int u = -1;
        int minDist = INT_MAX;

        // Array-based priority queue:
        // find the unvisited vertex with the smallest distance
        for (int i = 0; i < V; i++)
        {
            if (!visited[i] && dist[i] < minDist)
            {
                minDist = dist[i];
                u = i;
            }
        }

        // If no reachable unvisited vertex remains, stop
        if (u == -1)
            break;

        visited[u] = true;

        // Scan the whole row of the adjacency matrix
        for (int v = 0; v < V; v++)
        {
            // graph[u][v] != INT_MAX means there is an edge
            if (!visited[v] && graph[u][v] != INT_MAX && dist[u] != INT_MAX)
            {
                if (dist[u] + graph[u][v] < dist[v])
                {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
    }

    return dist;
}
```

Adjacency Matrix + Array

Dijkstra B

```cpp
// part b
// stored : adjacency list, pq: min heap
vector<int> dijkstra_b(vector<vector<pair<int, int>>> &adj, int src)
{

    int V = adj.size();

    // Min-heap (priority queue) storing pairs of (distance, node)
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

    vector<int> dist(V, INT_MAX);

    // Distance from source to itself is 0
    dist[src] = 0;
    pq.emplace(0, src);

    // Process the queue until all reachable vertices are finalized
    while (!pq.empty())
    {
        auto top = pq.top();
        pq.pop();

        int d = top.first;
        int u = top.second;

        // If this distance not the latest shortest one, skip it
        if (d > dist[u])
            continue;

        // Explore all neighbors of the current vertex
        for (auto &p : adj[u])
        {
            int v = p.first;
            int w = p.second;

            // If we found a shorter path to v through u, update it
            if (dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                pq.emplace(dist[v], v);
            }
        }
    }

    // Return the final shortest distances from the source
    return dist;
}
```

Adjacency List + Min Heap

2 Factors that Differ: Graph Representation

Dijkstra A

	0	1	2	3
0	0	4	0	7
1	4	0	2	0
2	0	2	0	3
3	7	0	3	0

Adjacency **Matrix**

Dijkstra B

$$0 \rightarrow (1,4), (3,7)$$

$$1 \rightarrow (0,4), (2,2)$$

$$2 \rightarrow (1,2), (3,3)$$

$$3 \rightarrow (0,7), (2,3)$$

Adjacency **List**

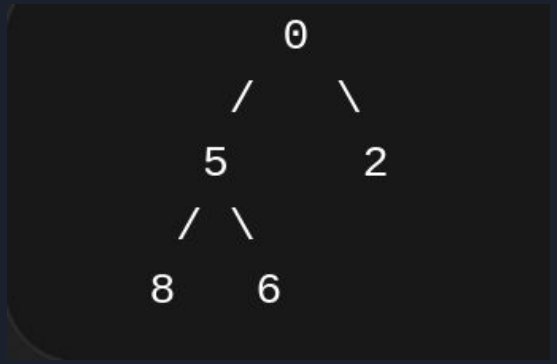
2 Factors that Differ: Priority Queue

Dijkstra A

```
Array:
[0] [5] [2] [8] [6]
 ^   ^   ^   ^   ^
check all entries to find minimum
```

Array

Dijkstra B



Min Heap

Data Generated

```
vector<int> sizes = {100, 200, 400, 800, 1600, 3200, 6400, 12800, 25600};
```

```
// generate a random dense adjacency matrix (every pair of nodes connected)
vector<vector<int>> generateDenseMatrix(int V)
{
    vector<vector<int>> g(V, vector<int>(V, INT_MAX));
    for (int i = 0; i < V; i++)
    {
        g[i][i] = 0;
        for (int j = i + 1; j < V; j++)
        {
            int w = rand() % 100 + 1;
            g[i][j] = g[j][i] = w;
        }
    }
    return g;
}
```

Dense

```
// generate a random sparse adjacency matrix (each node has around 5 neighbours)
vector<vector<int>> generateSparseMatrix(int V)
{
    vector<vector<int>> g(V, vector<int>(V, INT_MAX));
    for (int i = 0; i < V; i++)
    {
        g[i][i] = 0;
        for (int k = 0; k < 5; k++)
        {
            int j = rand() % V;
            if (i != j)
            {
                int w = rand() % 100 + 1;
                g[i][j] = g[j][i] = w;
            }
        }
    }
    return g;
}
```

Sparse

Theoretical Time Complexity

Adjacency Matrix + Array

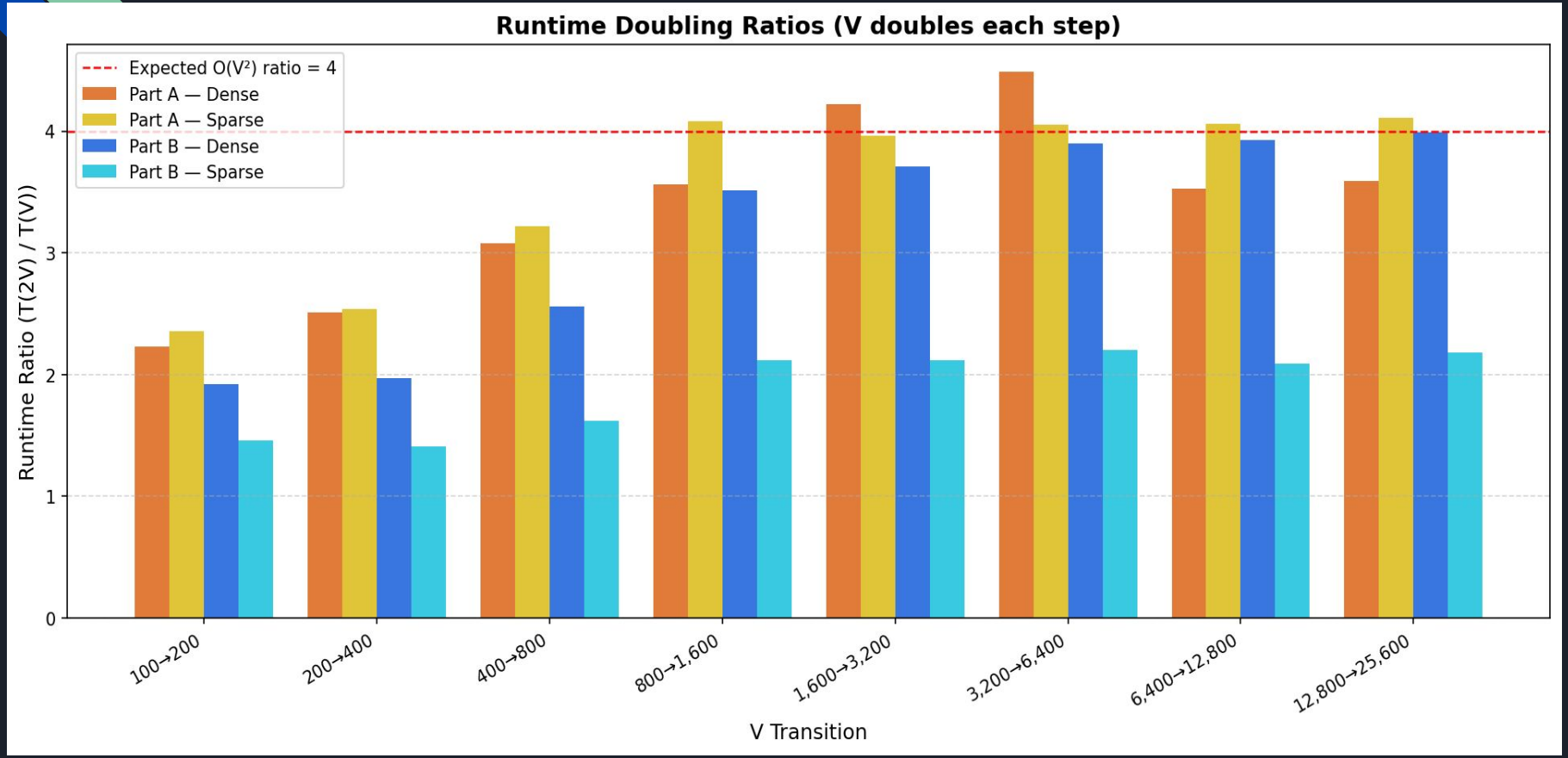
- Finding minimum distance vertex (linear scan):
  - $O(|V|)$ complexity per vertex
  - $O(|V|^2)$ for V vertices
- Relax edges:
  - Check all possible neighbours, $O(|V|)$ per vertex
  - $O(|V|^2)$ for V vertices
- **Total Time Complexity: $O(|V|^2)$**
- Thus, even for graph with fewer edges, adjacency matrix forces checking all V possible edges
  - Complexity depends only on |V|, not |E|

Theoretical Time Complexity

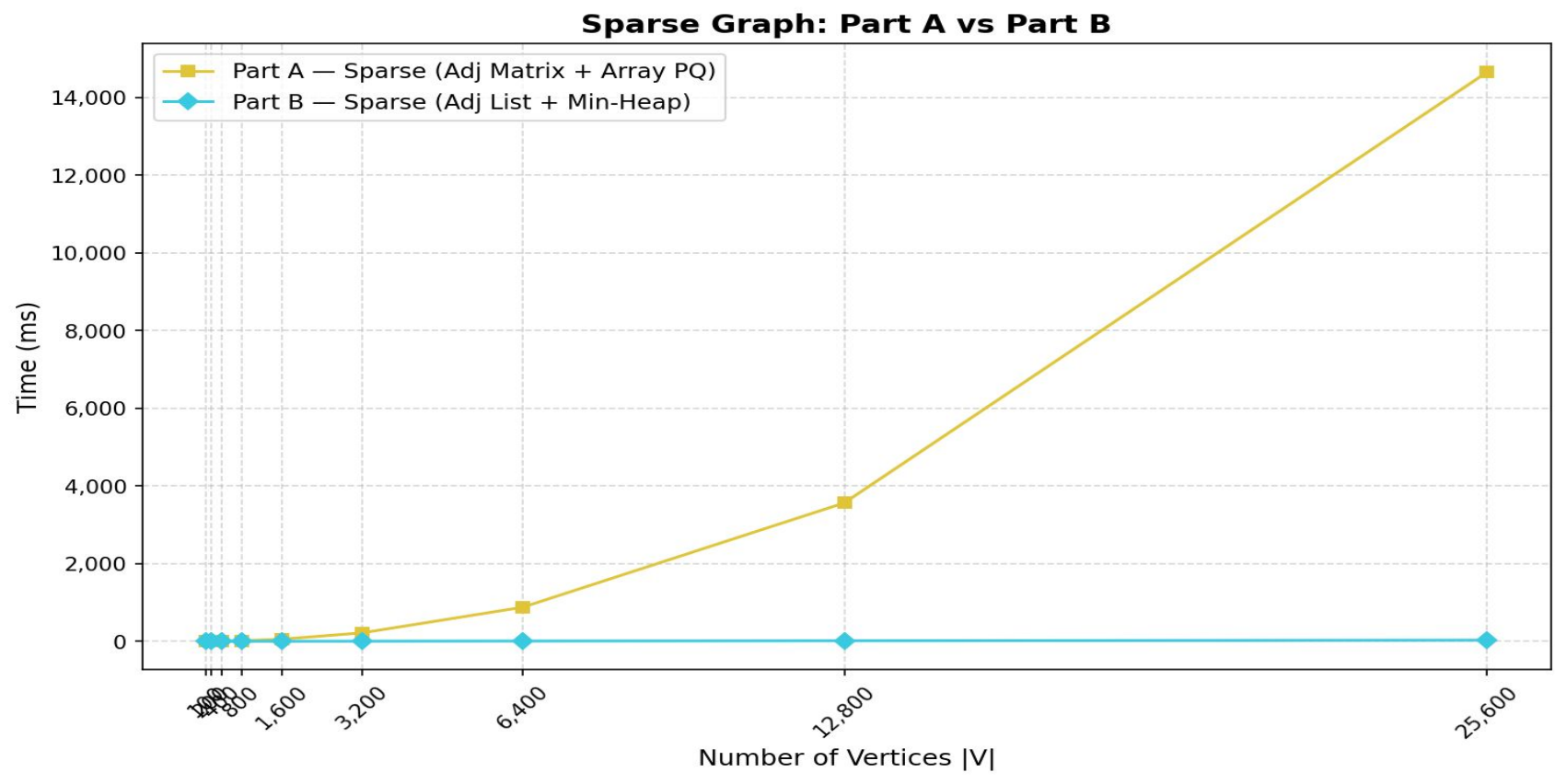
Adjacency List + Min-Heap

- Extract-Min (from heap):
  - O(log |V|) complexity per vertex
  - O(|V| log |V|) for V vertices
- Relax edges (Decrease-Key):
  - Each edge may trigger a decrease-key
  - O(log |V|) per operation
  - Total O(|E| log |V|)
- **Total Time Complexity: O((|V| + |E|) log |V|)** ~= O(|E| log |V|)
- Efficient because only actual edges are processed
- Much faster for sparse graphs where |E| is much less than $|V|^2$

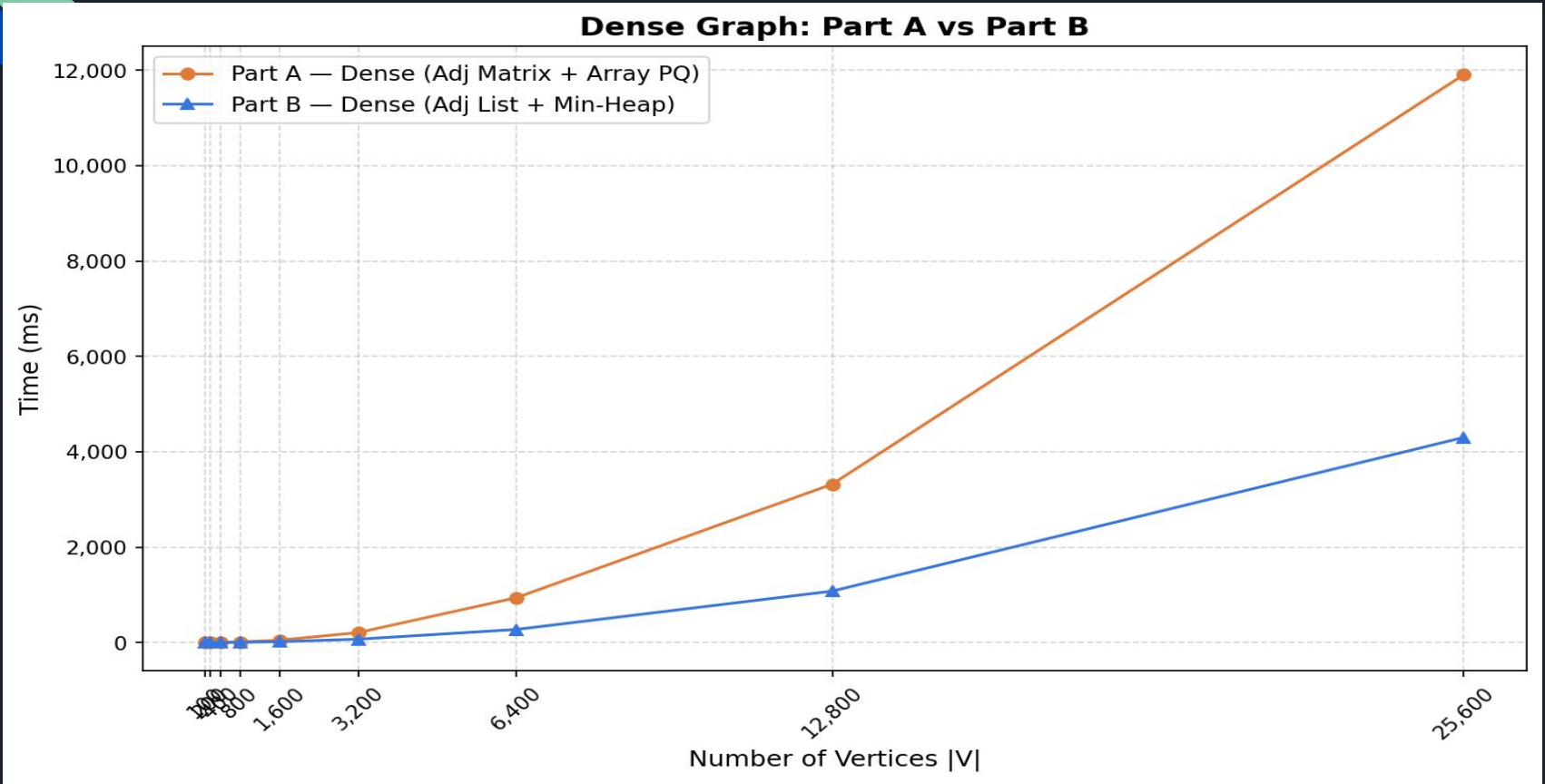
Empirical Time Analysis



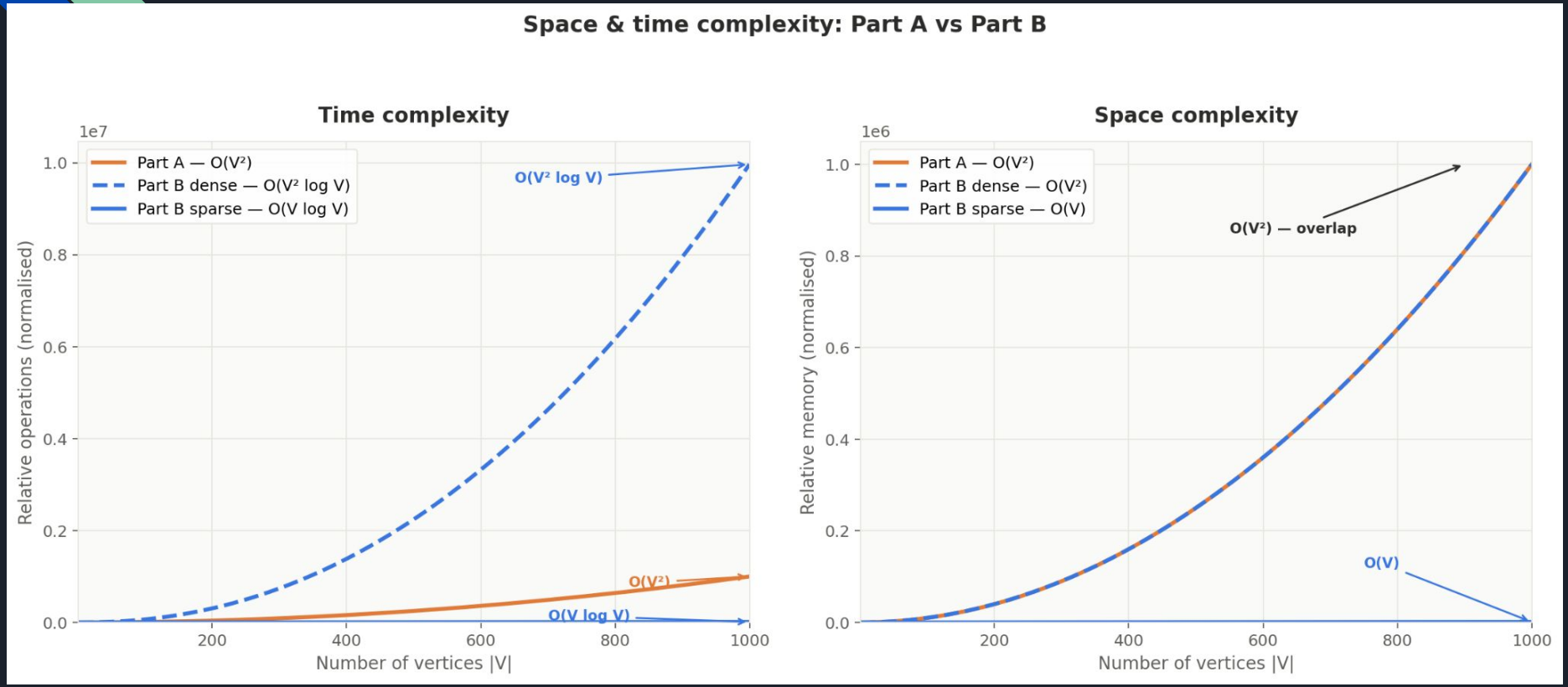
Empirical Time Analysis



Empirical Time Analysis



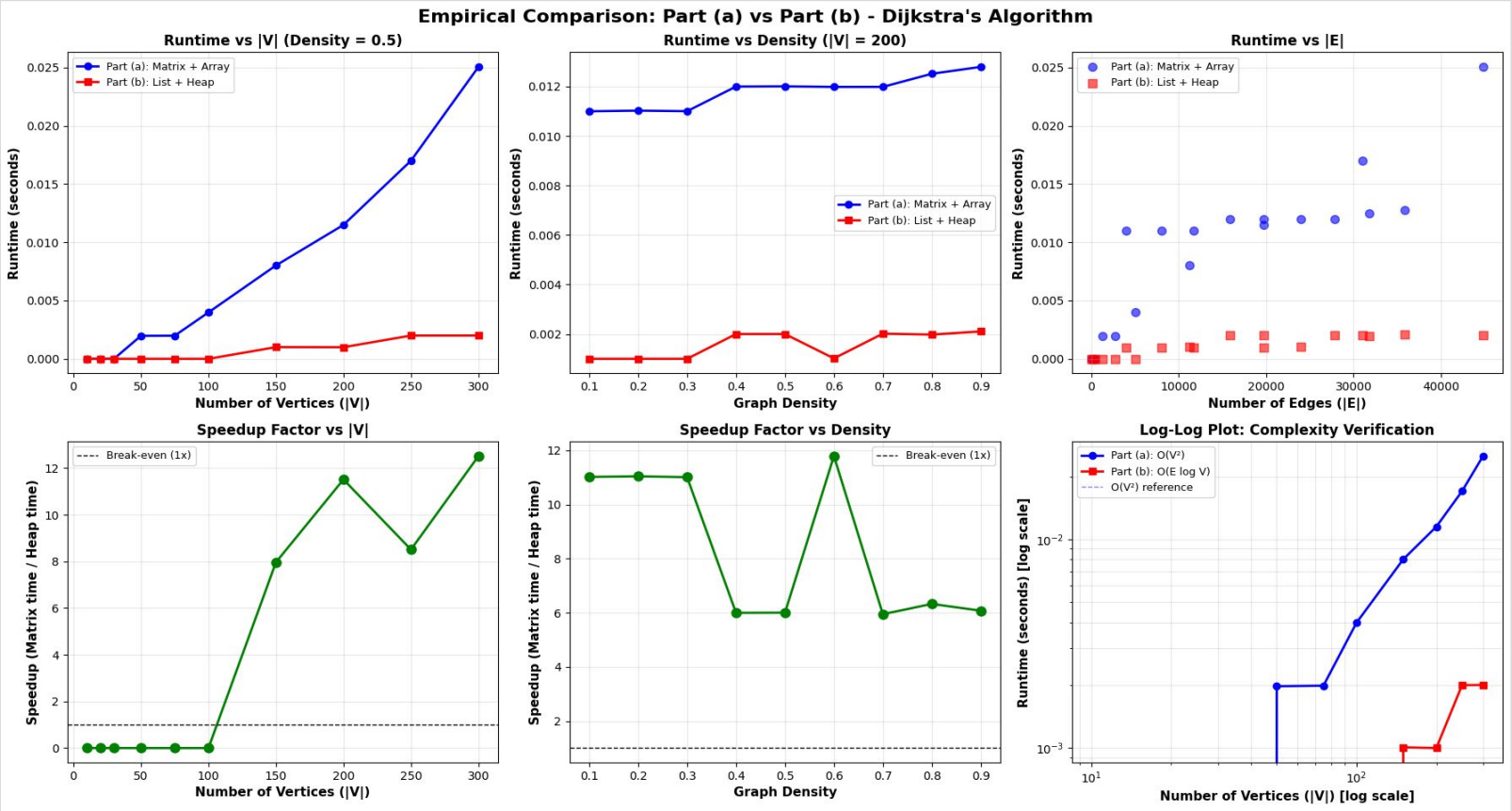
Empirical Space Analysis



Compare the two implementations in (a) and (b).

	Adjacency Matrix + Array-based Queue	Adjacency List + Min-Heap
Representation	Every vertex pair is represented regardless of whether an edge exists	Non-edges are omitted and not stored
Selection of next Vertex	Scans all unvisited vertices for minimum tentative distance each iteration	Delegates selection to heap operations instead of full scan in outer loop
Time Complexity	$O(V^2)$	$O((V+E) \log V)$
Space complexity	$O(V^2)$	$O(V+E)$

Graphs for Comparison between (a) and (b)



Time Complexity Comparison

Part (a) (Matrix + Array PQ):

Empirical: Runtime depends only on **|V|**, not |E|

Example: |V|=300, density=0.5 → **0.0251s**

Part (b) (Adjacency List + Min-Heap):

Empirical: Runtime depends on both **|V| and |E|**

Example: |V|=300, density=0.5 → **0.0020s**

Performance Comparison

- Part (b) is **consistently faster** for larger graphs
  Maximum observed speedup:
- **Roughly 12.52× faster at |V|=300**

- For small graphs (e.g., |V|=10):
  - **Negligible difference**
- Part (b) shows **better scalability**

Density Impact

Sparse graph (density = 0.1):

- |E| = 4049
- Part (b) is **11.02× faster**

Dense graph (density = 0.9):

- |E| = 35821
- Part (b) is **6.08× faster**

Trend Observed

Increasing density means **reduced speed advantage of Part (b) because** more edges will lead to more heap operations

Key Observations

Part (a) runtime is **independent of |E|**

Part (b) runtime **increases with |E| and outperforms Part (a)** for sparse and medium graphs

Performance gap **narrows as density increases**

Part (a) for small or very dense graphs where simplicity is sufficient.

Part (b) for large, sparse-to-moderate graphs where optimal real-world performance is required.

Thank You